

INTERVIEW PREPARATION GUIDE

.NET Full Stack Developer

ইন্টারভিউ প্রস্তুতি গাইড

সম্পূর্ণ বাংলায় — প্রশ্ন ও বিস্তারিত উত্তর

কোম্পানি

iTechVelocity (Edison Technologies)

পদ

Full Stack Developer (.NET)

আবেদনের শেষ তারিখ

২১ মে ২০২৬

অবস্থান

Niketan, Dhaka

80+

প্রশ্ন ও উত্তর

৭

বিভাগ



ইন্টারভিউ টিপস

বাংলা

সম্পূর্ণ গাইড



বিষয়সূচি

নিচের ৭টি বিভাগে সাজানো হয়েছে iTechVelocity-র job description অনুযায়ী

১

.NET Core ও C# বেসিক

৪টি প্রশ্ন — Framework পার্থক্য, DI, Interface, async/await



২

ASP.NET Core Web API

৪টি প্রশ্ন — REST, Middleware, JWT, Exception Handling



৩

MS SQL Server ও Database

৫টি প্রশ্ন — Stored Procedure, Index, Query Optimization, Transaction, CTE



৪

Entity Framework Core

৩টি প্রশ্ন — Code First, Loading strategies, LINQ



৫

Redis ও Caching

৩টি প্রশ্ন — In-Memory Cache, Redis, Cache Patterns



৬

Clean Architecture ও SOLID

৪টি প্রশ্ন — Architecture layers, SOLID, Repository, DDD



৭

সাধারণ HR ও Behavioral প্রশ্ন

৪টি প্রশ্ন — নিজেকে উপস্থাপন, চ্যালেঞ্জ, দুর্বলতা, Salary



বিভাগ ১ — .NET Core ও C# বেসিক

সবচেয়ে সাধারণ প্রশ্ন যা প্রায় সব interview-এ আসে

প্রশ্ন ০১

.NET Core এবং .NET Framework-এর মধ্যে পার্থক্য কী?

উত্তর

.NET Framework শুধুমাত্র Windows-এ চলে এবং এটি পুরনো প্ল্যাটফর্ম। এটি legacy enterprise application-এ ব্যবহৃত হয়।

.NET Core (এখন .NET 5/6/7/8+) একটি cross-platform, open-source ফ্রেমওয়ার্ক যা Windows, Linux এবং macOS সব জায়গায় চলে। এটি দ্রুত, হালকা এবং modular। আধুনিক Web API, microservice এবং cloud application তৈরিতে .NET Core ব্যবহার করা হয়।

মূল পার্থক্য: Cross-platform support, performance, open-source nature এবং Docker সহ cloud-native deployment।

 **ইন্টারভিউ টিপস:** বলুন — "iTechVelocity-তে আমি .NET Core ব্যবহার করে scalable ERP API তৈরি করতে চাই। আমি .NET 7/8-এর latest features যেমন minimal API এবং performance improvements সম্পর্কে জানি।"

প্রশ্ন ০২

Dependency Injection (DI) কী এবং .NET Core-এ কীভাবে কাজ করে?

উত্তর

Dependency Injection হলো একটি design pattern যেখানে একটি class তার নিজের dependency তৈরি না করে বাইরে থেকে inject করে নেয়। এটি code-কে *loosely coupled* এবং testable করে।

.NET Core-এ built-in DI container আছে। `Program.cs` -এ তিনভাবে register করা যায়:

- `AddScoped` — প্রতিটি HTTP request-এ একটি instance তৈরি হয়
- `AddTransient` — প্রতিবার inject করলে নতুন instance তৈরি হয়
- `AddSingleton` — পুরো application-এ একটিই instance থাকে

Constructor injection সবচেয়ে বেশি ব্যবহৃত হয়।

💡 **ইন্টারভিউ টিপস:** AddScoped, AddTransient, AddSingleton-এর পার্থক্য অবশ্যই মুখস্থ রাখুন। ERP-এ Database service সাধারণত Scoped করা হয়।

প্রশ্ন ০৩

Interface এবং Abstract Class-এর পার্থক্য কী?

উত্তর

Interface: শুধু method signature থাকে, কোনো implementation থাকে না (C# 8+ থেকে default implementation সম্ভব)। একটি class একাধিক interface implement করতে পারে।
যেমন: `IRepository<T>`

Abstract Class: Partial implementation থাকতে পারে। একটি class শুধুমাত্র একটি abstract class inherit করতে পারে।

কখন কোনটি ব্যবহার করবেন?

- Multiple inheritance দরকার হলে → Interface
- Shared implementation share করতে হলে → Abstract Class
- SOLID-এর Interface Segregation Principle মেনে চলতে interface বেশি পছন্দনীয়।

💡 **ইন্টারভিউ টিপস:** Repository Pattern-এ `IRepository` interface-এর উদাহরণ দিন। এটি interview-এ practical knowledge দেখায়।

প্রশ্ন ০৪

async/await কী এবং কেন ব্যবহার করতে হয়?

উত্তর

async/await হলো asynchronous programming-এর জন্য C#-এর keyword। যখন কোনো কাজ I/O bound (যেমন database query, HTTP call, file read) তখন async method ব্যবহার করা উচিত।

এটি thread block না করে অন্য কাজ করতে দেয়, ফলে API-র performance এবং scalability ভালো হয়।

সতর্কতা:

- `async void` এড়িয়ে চলুন — সবসময় `async Task` ব্যবহার করুন

- CPU-bound কাজে `Task.Run()` ব্যবহার করুন
- Deadlock এড়াতে `.Result` বা `.Wait()` ব্যবহার করবেন না

💡 **ইন্টারভিউ টিপস:** "Synchronous call server-এর thread আটকে রাখে, কিন্তু async/await দিয়ে একই সময়ে আরো বেশি request handle করা যায়" — এটি বললে senior developer-এর মতো শোনাবে।

বিভাগ ২ — ASP.NET Core Web API

iTechVelocity-র job description-এর সবচেয়ে গুরুত্বপূর্ণ বিষয়

প্রশ্ন ০৫

RESTful API-তে HTTP Methods গুলো কী এবং কোনটি কখন ব্যবহার হয়?

উত্তর

- **GET** — ডেটা পড়ার জন্য। যেমন: `GET /api/products` (সব পণ্য)
- **POST** — নতুন ডেটা তৈরির জন্য। যেমন: `POST /api/products`
- **PUT** — সম্পূর্ণ ডেটা replace করার জন্য। যেমন: `PUT /api/products/1`
- **PATCH** — আংশিক আপডেটের জন্য। যেমন: শুধু price পরিবর্তন
- **DELETE** — ডেটা মুছে ফেলার জন্য। যেমন: `DELETE /api/products/1`

গুরুত্বপূর্ণ Status Codes:

200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 500 Internal Server Error

💡 **ইন্টারভিউ টিপস:** Idempotency জানুন — GET, PUT, DELETE idempotent (বারবার call করলে একই result), কিন্তু POST idempotent নয়।

প্রশ্ন ০৬

Middleware কী এবং .NET Core API-তে কীভাবে কাজ করে?

উত্তর

Middleware হলো HTTP request pipeline-এর একটি component যা request আসার সময় এবং response যাওয়ার সময় কাজ করে। এগুলো chain-এর মতো একটির পর একটি execute হয়।

Built-in Middleware উদাহরণ:

- `UseAuthentication()` — Token verify করে
- `UseAuthorization()` — Permission check করে
- `UseCors()` — Cross-origin request handle করে
- `UseExceptionHandler()` — Global error catch করে

Custom Middleware দিয়ে IP blocking, request logging, machine locking করা যায় — যা iTechVelocity-র JD-তে উল্লেখ আছে।

💡 **ইন্টারভিউ টিপস:** Middleware-এর order গুরুত্বপূর্ণ — Authentication আগে, Authorization পরে আসতে হবে। এই detail জানলে interviewer impressed হবেন।

প্রশ্ন ০৭

JWT Authentication কীভাবে কাজ করে? বিস্তারিত বলুন।

উত্তর

JWT (JSON Web Token) তিন অংশে থাকে: `Header.Payload.Signature`

Flow:

১. User username/password দিয়ে `POST /api/auth/login` করে
২. Server credentials verify করে একটি JWT token তৈরি করে পাঠায়
৩. Client পরের প্রতিটি request-এ Header-এ পাঠায়: `Authorization: Bearer <token>`
৪. Server token verify করে (signature check) এবং access দেয়

Access Token vs Refresh Token:

- Access Token — স্বল্পমেয়াদী (15 মিনিট), API call-এ ব্যবহার হয়
- Refresh Token — দীর্ঘমেয়াদী (7 দিন), নতুন access token নিতে ব্যবহার হয়

💡 **ইন্টারভিউ টিপস:** JWT stateless — server-এ কোনো session রাখতে হয় না। এটি scalable ERP system-এর জন্য উপযুক্ত।

প্রশ্ন ০৮

Global Exception Handling কীভাবে implement করবেন?

উত্তর

প্রতিটি controller-এ try-catch না লিখে **Global Exception Middleware** তৈরি করুন।

পদ্ধতি:

১. `IMiddleware` implement করে custom middleware তৈরি করুন
২. সব exception catch করে consistent error response দিন
৩. `ILogger` দিয়ে error log করুন
৪. Production-এ sensitive error details hide করুন

Response Format (RFC 7807 ProblemDetails):

```
{ "status": 500, "title": "Server Error", "detail": "..." }
```

এটি code duplication কমায় এবং সব error একই format-এ আসে।

 **ইন্টারভিউ টিপস:** Serilog বা NLog দিয়ে structured logging করার কথা বলুন। Log-এ UserId, RequestId, Timestamp রাখা production debugging-এ সাহায্য করে।

৫. Interview এ বলার মতো Points

InvokeAsync() middleware এর main method
RequestDelegate next পরের middleware call করে
ILogger দিয়ে centralized logging করা যায়
Production এ sensitive data hide করা security best practice
Consistent response frontend handle করা সহজ করে

কেন IMiddleware ব্যবহার করবো?

IMiddleware ব্যবহার করলে:

Dependency Injection cleaner হয়
Middleware test করা সহজ হয়
Scoped service inject করা যায়
Large project এ maintain করা সহজ হয়

বিভাগ ৩ — MS SQL Server ও Database

iTechVelocity বিশেষভাবে complex stored procedure ও query optimization চেয়েছে

প্রশ্ন ০৯

Stored Procedure কী এবং কেন ব্যবহার করা হয়?

উত্তর

Stored Procedure হলো SQL Server-এ সংরক্ষিত SQL কোডের একটি সেট যা নাম দিয়ে বারবার call করা যায়।

সুবিধা:

- **Performance:** Execution plan cache হয়, বারবার compile করতে হয় না
- **Security:** Direct table access না দিয়ে SP-র মাধ্যমে access দেওয়া যায়
- **Maintainability:** Business logic একজায়গায় থাকে
- **Network Traffic কম:** একটি call-এ অনেক কাজ হয়

ERP system-এ payment processing, stock calculation, report generation-এ SP ব্যবহার করা হয়।

💡 **ইন্টারভিউ টিপস:** একটি SP লিখুন যা pagination সহ employee list ফেরত দেয়: `OFFSET @PageSize * (@PageNumber - 1) ROWS FETCH NEXT @PageSize ROWS ONLY`

প্রশ্ন ১০

Index কী? Clustered এবং Non-Clustered Index-এর পার্থক্য বলুন।

উত্তর

Index হলো database-এ data দ্রুত খোঁজার জন্য একটি data structure — বইয়ের সূচিপত্রের মতো।

Clustered Index:

- Table-এর actual data এই index-এর order অনুযায়ী physically সাজানো থাকে
- একটি table-এ শুধু একটি Clustered Index থাকতে পারে
- সাধারণত Primary Key-তে automatically তৈরি হয়

Non-Clustered Index:

- আলাদা structure যেখানে index value এবং actual data-র pointer থাকে
- একটি table-এ একাধিক Non-Clustered Index থাকতে পারে
- WHERE clause-এ বেশি ব্যবহৃত column-এ তৈরি করা হয়

💡 **ইন্টারভিউ টিপস:** বেশি index করলে SELECT দ্রুত হয় কিন্তু INSERT/UPDATE/DELETE slow হয় — এই trade-off বলতে পারলে impression ভালো হবে।

প্রশ্ন ১১

Query Optimization কীভাবে করবেন? Slow query কীভাবে ঠিক করবেন?

উত্তর

Step-by-step approach:

১. **Execution Plan দেখুন** — SSMS-এ Ctrl+M চাপুন, "Table Scan" বা "Index Scan" খুঁজুন
২. **SELECT * এড়িয়ে চলুন** — শুধু প্রয়োজনীয় column select করুন
৩. **WHERE clause-এ indexed column ব্যবহার করুন**
৪. **N+1 Query সমস্যা এড়ান** — loop-এর মধ্যে query করবেন না
৫. **Pagination ব্যবহার করুন** — বড় result set-এ OFFSET...FETCH ব্যবহার করুন
৬. **CTE ব্যবহার করুন** — জটিল nested query-কে readable করুন
৭. **Statistics আপডেট রাখুন** — `UPDATE STATISTICS TableName`

💡 **ইন্টারভিউ টিপস:** `SET STATISTICS IO ON` দিয়ে logical reads দেখুন। Optimization-এর আগে ও পরে এই number compare করুন।

প্রশ্ন ১২

Transaction কী এবং ACID Properties কী?

উত্তর

Transaction হলো SQL operations-এর একটি unit — সব একসাথে সফল হয় অথবা কোনোটিই হয় না।

ACID Properties:

- **A — Atomicity:** সব operation সম্পন্ন হবে অথবা কোনোটিই হবে না
- **C — Consistency:** Database সবসময় valid state-এ থাকবে
- **I — Isolation:** Concurrent transaction একে অপরকে প্রভাবিত করবে না
- **D — Durability:** Committed transaction permanent, power off হলেও থাকবে

`BEGIN TRANSACTION` , `COMMIT` , `ROLLBACK` দিয়ে control করা হয়।

💡 **ইন্টারভিউ টিপস:** ERP-এ payment বা stock update-এ transaction ব্যবহার করতে হয় — bank account থেকে টাকা কাটা এবং অন্য account-এ যোগ করা একই transaction-এ হওয়া উচিত।

প্রশ্ন ১৩

CTE (Common Table Expression) কী এবং কখন ব্যবহার করবেন?

উত্তর

CTE হলো একটি temporary named result set যা একটি query-র মধ্যে define করা হয় এবং সেই query-তে ব্যবহার করা যায়। `WITH` keyword দিয়ে শুরু হয়।

সুবিধা:

- Complex nested query-কে readable করে
- Recursive query লেখা যায় (যেমন hierarchy/tree structure)
- একই subquery বারবার লিখতে হয় না

উদাহরণ: Organization hierarchy (Manager → Employee → Sub-employee) বের করতে Recursive CTE ব্যবহার করা হয়।

💡 **ইন্টারভিউ টিপস:** CTE vs Subquery vs Temp Table — তিনটির পার্থক্য জানুন। CTE সবচেয়ে readable, Temp Table large data-এ ভালো।

বিভাগ ৪ — Entity Framework Core

Database access layer-এর জন্য ORM tool

প্রশ্ন ১৪

Code First এবং Database First approach কী? কোনটি কখন ব্যবহার করবেন?

উত্তর

Code First:

প্রথমে C# class (Entity) তৈরি করুন, তারপর EF Core migration দিয়ে database automatically তৈরি হয়।

Commands: `Add-Migration InitialCreate` → `Update-Database`

নতুন project-এ এটি পছন্দনীয়।

Database First:

আগে database তৈরি করুন, তারপর EF Core scaffold দিয়ে C# class generate করা হয়।

Command: `Scaffold-DbContext "ConnectionString"`

`Microsoft.EntityFrameworkCore.SqlServer`

Legacy system-এ বা DBA-driven project-এ ব্যবহার হয়।

💡 **ইন্টারভিউ টিপস:** Migration-এর ইতিহাস Git-এ রাখুন। Production-এ `Update-Database` command দেওয়ার আগে `script generate` করুন: `Script-Migration`

প্রশ্ন ১৫

Eager Loading, Lazy Loading এবং Explicit Loading-এর পার্থক্য কী?

উত্তর

Eager Loading: `Include()` দিয়ে related data একসাথে load হয়। একটি JOIN query-তে সব data আসে। সবচেয়ে বেশি ব্যবহৃত।

```
db.Orders.Include(o => o.Customer).ToList()
```

Lazy Loading: Related data তখনই load হয় যখন access করা হয়। N+1 query সমস্যা তৈরি করতে পারে — সতর্ক থাকুন।

Explicit Loading: `Entry().Collection().Load()` দিয়ে manually load করা হয়। কখন load করবেন সেটা developer control করেন।

সুপারিশ: Performance-এর জন্য Eager Loading সবচেয়ে ভালো।

💡 **ইন্টারভিউ টিপস:** Lazy Loading enable করলে অজান্তে অনেক query হতে পারে। ERP-এ এটি এড়িয়ে Eager Loading বা Explicit Loading ব্যবহার করুন।

প্রশ্ন ১৬

LINQ কী? কিছু উদাহরণ দিন।

উত্তর

LINQ (Language Integrated Query) হলো C#-এ data query করার একটি পদ্ধতি যা SQL-এর মতো কিন্তু type-safe।

সাধারণ LINQ operations:

- `Where()` — ফিলটার করতে
- `Select()` — প্রয়োজনীয় field বেছে নিতে
- `OrderBy()` / `OrderByDescending()` — sort করতে
- `GroupBy()` — group করতে
- `FirstOrDefault()` — প্রথম element নিতে
- `Any()` / `All()` — condition check করতে
- `Sum()` / `Count()` / `Average()` — aggregate করতে

EF Core-এ LINQ query automatically SQL-এ convert হয়।

💡 **ইন্টারভিউ টিপস:** `AsNoTracking()` ব্যবহার করুন read-only query-তে — এটি performance বাড়ায় কারণ EF Core object track করে না।

বিভাগ ৫ — Redis ও Caching

Junior developer-রা কম জানে — এটি জানলে আপনি এগিয়ে থাকবেন

প্রশ্ন ১৭

Caching কেন দরকার এবং কত ধরনের Caching আছে?

উত্তর

Caching database-এর load কমায় এবং API response দ্রুত করে। বারবার একই data database থেকে না এনে memory থেকে দেওয়া হয়।

ধরন:

- **In-Memory Cache:** একই server-এর memory-তে data রাখে। Single server application-এ ভালো। `IMemoryCache` interface ব্যবহার হয়।
- **Distributed Cache (Redis):** আলাদা cache server। Multiple server বা microservice-এ ব্যবহার হয়। Server restart-এ data থাকে।
- **Output Cache:** পুরো HTTP response cache করে। Static বা semi-static page-এ ব্যবহার হয়।

Cache করার সময় **TTL (Time To Live)** এবং **Cache Invalidation strategy** ঠিক করা জরুরি।

💡 **ইন্টারভিউ টিপস:** Cache-aside pattern সবচেয়ে সাধারণ — "cache-এ না থাকলে DB থেকে আনো এবং cache-এ রাখো।" এটি বলুন।

প্রশ্ন ১৮

Redis কী এবং .NET Core-এ কীভাবে implement করবেন?

উত্তর

Redis হলো একটি open-source, in-memory data store যা key-value pair হিসেবে data রাখে। এটি অত্যন্ত দ্রুত — microsecond-এ response দেয়।

ব্যবহার: Distributed caching, session management, pub/sub messaging, rate limiting।

.NET Core-এ implement করুন:

১. Package: `StackExchange.Redis`

২. Program.cs -এ: `services.AddStackExchangeRedisCache(...)`
৩. `IDistributedCache` inject করুন
৪. `GetStringAsync(key)` দিয়ে পড়ুন, `SetStringAsync(key, value)` দিয়ে লিখুন

Data serialize করে JSON string হিসেবে রাখুন।

💡 **ইন্টারভিউ টিপস:** Docker দিয়ে local-এ Redis run করুন: `docker run -d -p 6379:6379 redis` — তারপর একটি demo project তৈরি করে portfolio-তে রাখুন।

প্রশ্ন ১৯

Cache Invalidation কী এবং এটি কেন কঠিন?

উত্তর

Cache Invalidation হলো cache-এ থাকা পুরনো data মুছে ফেলা বা আপডেট করা যখন মূল data পরিবর্তন হয়।

Strategies:

- **TTL (Time-based):** নির্দিষ্ট সময় পর cache automatically expire হয়। সহজ কিন্তু stale data দেখাতে পারে।
- **Event-based:** Data update হলে manually cache delete করো। `cache.Remove(key)`
- **Write-through:** Data write করার সময় একসাথে DB এবং cache আপডেট হয়।

Computer science-এর বিখ্যাত উক্তি: "There are only two hard things in CS: cache invalidation and naming things."

💡 **ইন্টারভিউ টিপস:** ERP-এ product price cache করলে price update-এর সাথে সাথে cache clear করতে হবে — event-based invalidation ব্যবহার করুন।

বিভাগ ৬ — Clean Architecture ও SOLID Principles

Senior developer-দের মতো চিন্তা করার ভিত্তি

প্রশ্ন ২০

Clean Architecture কী এবং layers কীভাবে সাজানো হয়?

উত্তর

Clean Architecture (Robert C. Martin) code-কে maintainable, testable এবং framework-independent করে।

৪টি Layer (ভেতর থেকে বাইরে):

- ① **Domain Layer:** Entities, Value Objects, Business Rules — কোনো dependency নেই
- ② **Application Layer:** Use Cases, Interfaces, DTOs, CQRS Commands/Queries
- ③ **Infrastructure Layer:** DB implementation, External API, Email service
- ④ **Presentation Layer:** API Controllers, Blazor UI

মূল নিয়ম (Dependency Rule): Inner layer জানে না outer layer সম্পর্কে। Domain কখনো EF Core জানবে না।

💡 **ইন্টারভিউ টিপস:** GitHub-এ একটি Clean Architecture project তৈরি করুন এবং interview-এ দেখান। এটি আপনাকে অন্য candidates থেকে আলাদা করবে।

প্রশ্ন ২১

SOLID Principles বিস্তারিত বলুন — প্রতিটির উদাহরণ দিন।

উত্তর

S — Single Responsibility Principle: একটি class-এর শুধু একটি কারণে পরিবর্তন হওয়া উচিত। UserService শুধু user business logic রাখবে, email পাঠানো আলাদা class-এ।

O — Open/Closed Principle: Class নতুন feature-এ extend করা যাবে কিন্তু existing code modify করা উচিত নয়। নতুন payment method যোগে existing class পরিবর্তন নয়, নতুন class তৈরি করুন।

L — Liskov Substitution Principle: Subclass তার parent-এর জায়গায় ব্যবহারযোগ্য হতে হবে, behavior নষ্ট না করে।

I — Interface Segregation Principle: বড় interface-এর বদলে ছোট specific interface ব্যবহার করুন। `IReadRepository` ও `IWriteRepository` আলাদা রাখুন।

D — Dependency Inversion Principle: High-level module concrete class-এ নয়, abstraction (interface)-এ নির্ভর করবে।

 **ইন্টারভিউ টিপস:** প্রতিটি principle-এর জন্য একটি করে real ERP code example প্রস্তুত রাখুন। Abstract বলার চেয়ে concrete example অনেক বেশি প্রভাব ফেলে।

প্রশ্ন ২২

Repository Pattern কী এবং কেন ব্যবহার করবেন?

উত্তর

Repository Pattern data access logic-কে business logic থেকে আলাদা করে।

Implementation:

- ① Generic interface তৈরি: `IRepository<T>` — GetAll, GetById, Add, Update, Delete
- ② EF Core implementation: `EfRepository<T> : IRepository<T>`
- ③ DI-তে register: `services.AddScoped(typeof(IRepository<>), typeof(EfRepository<>))`

সুবিধা:

- Database পরিবর্তন করলে শুধু repository change, business logic অপরিবর্তিত
- Unit testing সহজ — mock repository দিয়ে test করা যায়
- Code duplication কমে

 **ইন্টারভিউ টিপস:** Unit of Work Pattern-এর সাথে মিলিয়ে ব্যবহার করুন — একাধিক repository-র operation একটি transaction-এ রাখতে UnitOfWork ব্যবহার হয়।

প্রশ্ন ২৩

Domain-Driven Design (DDD) কী? Basic concepts বলুন।

উত্তর

DDD হলো complex business domain-কে software design-এ reflect করার একটি approach।

মূল concepts:

- **Entity:** Unique identity আছে এমন object। যেমন: Customer (CustomerId দিয়ে identify হয়)
- **Value Object:** Identity নেই, value দিয়ে define হয়। যেমন: Address, Money
- **Aggregate:** Related entity-র একটি cluster। Aggregate Root এর মাধ্যমে access হয়
- **Repository:** Aggregate-কে persist করার উপায়
- **Domain Event:** Business-এ কিছু ঘটলে event fire হয়। যেমন: OrderPlaced, PaymentReceived
- **Bounded Context:** Domain-এর একটি boundary যেখানে নির্দিষ্ট model ব্যবহার হয়

💡 **ইন্টারভিউ টিপস:** DDD-র সব detail না জানলেও মূল concepts বললেই যথেষ্ট। "আমি DDD concepts অনুসরণ করে Entity ও Value Object আলাদা রাখি" — এটুকু বললে ভালো impression হয়।

বিভাগ ৭ — সাধারণ HR ও Behavioral প্রশ্ন

Technical দক্ষতার পাশাপাশি এই প্রশ্নগুলো interview-এ গুরুত্বপূর্ণ ভূমিকা রাখে

প্রশ্ন ২৪

নিজেকে পরিচয় করিয়ে দিন — "Tell me about yourself"

উত্তর — Template

কাঠামো (১-২ মিনিট):

"আমি [নাম], [University] থেকে Computer Science/Engineering-এ graduate। আমার [X মাস/বছর] .NET development-এর অভিজ্ঞতা আছে।

আমি ASP.NET Core Web API, Entity Framework Core এবং MS SQL Server নিয়ে কাজ করেছি। সম্প্রতি আমি একটি [আপনার project] তৈরি করেছি যেখানে Clean Architecture এবং JWT Authentication ব্যবহার করেছি।

আমি ERP system-এ আগ্রহী কারণ এটি real business problem solve করে। iTechVelocity-তে এসে আমি দ্রুত শিখতে এবং দলের সাথে quality software তৈরি করতে চাই।"

 **ইন্টারভিউ টিপস:** আগে থেকে practice করুন — mirror-এর সামনে বা phone-এ record করে শুনুন। প্রথম ১ মিনিটে strong impression তৈরি করুন।

প্রশ্ন ২৫

আপনার সবচেয়ে চ্যালেঞ্জিং project কোনটি ছিল?

উত্তর — STAR Method অনুসরণ করুন

STAR Method: Situation → Task → Action → Result

উদাহরণ উত্তর:

Situation: "আমি একটি Inventory Management system তৈরি করছিলাম।"

Task: "Real-time stock update এবং complex report generation করতে হতো।"

Action: "প্রথমে query অনেক slow ছিল। Execution plan analyze করে index add করেছি এবং N+1 query সমস্যা ঠিক করেছি। Redis দিয়ে frequently accessed data cache করেছি।"

Result: "API response time 3 সেকেন্ড থেকে 300 millisecond-এ নামিয়ে এনেছি — ১০ গুণ দ্রুত।"

💡 **ইন্টারভিউ টিপস:** সবসময় **measurable result** দিন — "faster" নয়, "৭০% faster"। Numbers বললে বিশ্বাসযোগ্যতা বাড়ে।

প্রশ্ন ২৬

আপনার দুর্বলতা কী? — "What is your weakness?"

উত্তর

সংভাবে বলুন কিন্তু দেখান যে আপনি actively improve করছেন।

উদাহরণ উত্তর:

"Backend API development-এ আমার experience তুলনামূলক কম। তাই আমি প্রতিদিন একটি করে mini project তৈরি করছি এবং Microsoft-এর official documentation follow করছি।

Redis আমার কাছে নতুন ছিল, কিন্তু আমি ইতিমধ্যে Docker-এ Redis চালু করে .NET Core-এ caching implement করেছি।

আমি believe করি weakness সম্পর্কে সচেতন থাকা এবং সেটা নিয়ে কাজ করা একজন ভালো developer-এর লক্ষণ।"

💡 **ইন্টারভিউ টিপস:** "আমার কোনো দুর্বলতা নেই" — এটি কখনো বলবেন না। দুর্বলতা স্বীকার করুন, কিন্তু সাথে improvement plan দিন।

প্রশ্ন ২৭

Salary negotiation কীভাবে করবেন?

উত্তর

JD-তে লেখা আছে "Negotiable (Market Standard)" — তাই আপনাকে research করতে হবে।

Dhaka-তে Junior .NET Developer-এর বাজারদর (2025-2026):

- Less than 1 বছর অভিজ্ঞতা: ২৫,০০০ — ৪৫,০০০ টাকা
- 1-2 বছর অভিজ্ঞতা: ৪০,০০০ — ৬৫,০০০ টাকা

Negotiation Strategy:

১. প্রথমে range জানতে চান: "আপনাদের budget range কত?"
২. আপনার skills ও project দিয়ে justify করুন

৩. শুধু salary নয়, benefits, training, growth opportunity-ও বিবেচনা করুন

৪. First offer-এ সাথে সাথে রাজি না হওয়া স্বাভাবিক

 **ইন্টারভিউ টিপস:** "শুধু salary-র জন্য আসছি" — এটি বলবেন না। বলুন "আমি শেখার এবং grow করার সুযোগ চাই, salary একটি factor মাত্র।"